Group 1 — Project Progress Report 1

Port Authority of New York & New Jersey

Group Members & Contributions

Name	Contribution
Harshita Bajaj	• External datasets download • Initial cleaning in Python
Harshith Doli	• Data Cleaning • Data Integration
Khan Mazen Moid	• Data Sampling
Manjil Adhikari	• Exploratory Data Analysis
Meghana Lakshminarayana Swamy	• Overall project assistance • Compilation, Project Reporting and Presentation

Understanding the Business Goals

The Port Authority of New York and New Jersey aims to leverage historical traffic and external data to understand key factors driving traffic patterns, toll violations, and congestion across its major crossings — including the George Washington Bridge, Holland Tunnel, Lincoln Tunnel, Bayonne Bridge, Goethals Bridge, and Outerbridge Crossing.

Specific business objectives include:

- Identifying the top factors affecting daily facility usage — including weather conditions, gas prices, severe storm events, seasonal trends, and vehicle type mix.
- Measuring and analyzing toll violation patterns by facility, time period (year, month, week), and vehicle category to support enforcement and revenue planning.
- Understanding how traffic volumes and speeds respond to external events such as severe weather, holidays, and seasonal demand shifts.
- Analyzing the impact of 2025 congestion pricing on traffic patterns compared to prior years to detect behavioral shifts across facilities.
- Forecasting daily traffic volumes beyond 2025 using H2O.ai AutoML to identify the best predictive model, then replicating it in Python for deployment.

Tools We Plan to Use

Analysis & Modeling

- **Python (Anaconda / Jupyter Notebook) —** Primary tool for data cleaning, EDA, sampling, and building regression, classification, and forecasting models.
- **R —** Used for Exploratory Data Analysis including descriptive statistics, visualizations, and correlation analysis.
- **SQL (SQLiteOnline) —** Used to integrate all five datasets into a single master dataset through LEFT JOIN operations.
- **H2O.ai AutoML —** To automatically benchmark and select the best-performing forecasting model across multiple algorithms, which is then replicated in Python.
- **Power BI —** To build interactive dashboards visualizing traffic patterns, violation trends, weather impacts, and forecasting outcomes.

Model Deployment

- **Python (Advanced Modeling) —** Used to replicate the AutoML-selected model from scratch and prepare it for production deployment.
- **FastAPI —** API layer that exposes the trained model as a REST endpoint for real-time traffic volume predictions.
- **Docker —** Containerizes the model and FastAPI application into a portable, reproducible deployment unit.
- **Kubernetes (Minikube) —** Orchestrates the containerized application on a local cluster, demonstrating scalable and production-ready infrastructure.

Algorithms and Models Planned

- **Time Series Forecasting (ARIMA, Prophet, ETS) —** To predict daily traffic volumes, capture seasonality, and model the effect of holidays and external events.
- **Regression Analysis —** To quantify the influence of weather, gas prices, storm events, and vehicle mix on traffic volume and violation rates.
- **Classification Models —** To detect and predict toll violators based on payment behavior, facility type, and traffic conditions.
- **H2O.ai AutoML —** To evaluate and rank multiple forecasting algorithms automatically, with the winning model replicated in Python as required by the project guidelines.

Section 1 — Examination of Available Data Sources

Five datasets were examined for this project — two internal datasets provided by the Port Authority and three external datasets sourced from U.S. government agencies. Each dataset is assessed below from both a business and technical perspective. Two internal datasets were excluded due to misalignment with the forecasting objective.

A. Internal Datasets (Company-provided)

1. All Recorded Traffic ✓ Selected

Attribute	Detail
Format	Tab-separated .txt file
Raw Records	5,383,378 rows (hourly, lane-level)
Cleaned Records	29,334 rows (daily, facility-level after aggregation)
Columns Kept	DATE, TIME, FAC_B, TOTAL, CASH, EZPASS, VIOLATION, Autos, Small_T, Large_T, Buses, Yr
Derived Feature	ViolationRate = VIOLATION / TOTAL (per day, per facility)
Date Range	2015 – 2025
Special Characteristics	Very large file; lane-level structure; NULL strings for missing values; TIME in HHMM format

Business Perspective:

- Primary dataset — directly measures bridge and tunnel usage across all Port Authority facilities.
- Contains total vehicle counts, Cash vs. EZPass breakdown, and toll violations.
- Facility-level daily aggregation supports traffic forecasting, congestion analysis, and violation reporting by time interval.

Technical Perspective:

- Memory-efficient loading required — only necessary columns loaded using usecols in pandas.
- 'NULL' strings treated as missing values at load time using na_values=['NULL'] and filled with 0 for count fields.
- Lane-level rows aggregated to daily facility level via groupby on DATE + FAC_B.
- DATE converted from string to proper datetime using pd.to_datetime() with error coercion and invalid rows dropped.

2. Facility Mobility Speeds ✓ Selected

Attribute	Detail
Format	CSV file
Raw Records	193 rows (monthly, facility-level)
Cleaned Records	5,975 rows (daily, facility-level after expansion)
Columns Kept	FAC_B, DATE, Avg_Speed, Delta
Date Range	Limited — expanded and forward-filled to align with project window
Special Characteristics	Monthly frequency; facility codes require mapping; GWB requires splitting into 3 sub-facilities

Business Perspective:

- Provides a congestion indicator through average speed at each facility — lower speed = higher congestion.
- Delta (deviation from free-flow speed) serves as a direct congestion proxy.
- Used as an explanatory feature in regression and forecasting models.

Technical Perspective:

- Monthly data expanded to daily granularity via full calendar merge and forward-fill within each facility.
- Facility codes (BB, GB, HT, LT, OBX, GWB) mapped to traffic naming convention (Bayonne, Goethals, Holland, Lincoln, Outerbridge).
- GWB expanded into three separate rows per day — GWB Lower, GWB Upper, GWB PIP — to match traffic dataset structure. This is required for referential integrity on the DATE + FAC_B join key.

3. All Recorded PABT Passenger ✗ Excluded

- Terminal-specific bus passenger dataset — not aligned with bridge and tunnel facility forecasting.
- Different operational structure; would add join complexity without improving model accuracy for the target objective.

4. All Recorded PABT Bus ✗ Excluded

- Terminal-specific bus operations dataset — redundant with PABT Passenger and outside project scope.
- Excluded to keep the integration clean and focused on the five selected datasets.

B. External Datasets Incorporated

1. NOAA Daily Weather — Newark Liberty International Airport

Attribute	Detail
Source	NOAA NCEI — Climate Data Online (CDO)
Website	https://www.ncei.noaa.gov/cdo-web/
Dataset	GHCN-Daily (Global Historical Climatology Network)
Format	CSV — direct portal download, pre-structured
Records	4,018 rows (one row per day, no duplicates)
Date Range	2015-01-01 to 2025-12-31 — full coverage
Variables	PRCP (precipitation), SNOW (snowfall), SNWD (snow depth), TMAX (max temp), TMIN (min temp), AWND (avg wind speed)
Missing Values	AWND: 9 rows (0.2%); SNWD: 3 rows — both handled via forward-fill
Special Characteristics	Single station; Fahrenheit temperatures; no multi-station merging needed

Business Perspective:

- Weather is a primary driver of daily traffic — precipitation, snow, and extreme temperatures suppress or shift travel demand.
- Newark Airport station is centrally located relative to Port Authority facilities, providing a single representative measurement for the NY/NJ region.

Technical Perspective:

- Arrives pre-structured as one row per day — minimal transformation required.
- STATION and NAME columns dropped as they are constant across all rows.
- JOIN KEY: DATE — merges directly with the daily traffic dataset.

2. NOAA Storm Events — Severe Weather Flag

Attribute	Detail
Source	NOAA NCEI — Storm Events Database
Website	https://www.ncei.noaa.gov/pub/data/swdi/stormevents/csvfiles/
Format	11 yearly CSV files (StormEvents_details FTP), merged into one
Records (Cleaned)	4,018 rows (one row per day after daily aggregation)
States Filtered	New York and New Jersey only
Event Types	Blizzard, Winter Storm, Heavy Snow, Ice Storm, Hurricane, Tropical Storm, Coastal Flood, Flood, Nor'easter, Storm Surge, High Wind
Derived Features	SevereFlag (0/1 binary) and SevereEventCount (integer count per day)
Severe Days	483 of 4,018 days (~12%) flagged as severe
Max Events in One Day	138 events (major multi-county storm impact)
Special Characteristics	Data covers through Sept 2025; Oct–Dec 2025 defaulted to 0

Business Perspective:

- Captures discrete named storm events that cause abnormal traffic shocks not reflected in continuous weather variables.
- SevereEventCount allows models to capture storm intensity — a day with 1 event vs. 138 has very different traffic impact.

Technical Perspective:

- 11 CSV files combined automatically using glob and pd.concat() — no manual merging.
- Event-level records aggregated to daily counts using groupby on date.
- Full 2015–2025 calendar created; non-storm days filled with SevereFlag = 0 and SevereEventCount = 0.
- JOIN KEY: DATE — merges directly with the daily traffic and weather datasets.

3. EIA Weekly Gas Prices — East Coast (PADD 1)

Attribute	Detail
Source	U.S. Energy Information Administration (EIA)
Website	https://www.eia.gov/petroleum/gasdiesel/
Dataset	Weekly Retail Gasoline Prices — East Coast PADD 1 region
Format	CSV — weekly observations, converted to daily
Records (Cleaned)	3,895 rows (daily after forward-fill expansion)
Date Range	2015-05-04 to 2025-12-31
Gas_Price Range	$1.79 – $4.55 per gallon (mean: $2.92)
Known Gap	Jan – Apr 2015 missing (no weekly observation before May 4, 2015)
Special Characteristics	Captures COVID-era lows (2020) and 2022 price spike — high-variance period for model training

Business Perspective:

- Gas prices directly influence driving frequency and discretionary travel — higher prices reduce toll usage, lower prices increase it.
- East Coast PADD 1 region reflects the actual price environment for NY/NJ metropolitan area drivers.

Technical Perspective:

- Weekly prices expanded to daily granularity via calendar merge and forward-fill — each day inherits the most recent weekly price.
- Jan–Apr 2015 gap noted as a known limitation — records before May 2015 excluded from the integrated dataset.
- JOIN KEY: DATE — merges directly with the daily traffic and weather datasets.

Section 2 — Exploratory Data Analysis (EDA)

EDA was performed in R using ggplot2, dplyr, corrplot, and psych. All five cleaned datasets were merged into one master dataframe before analysis. The tool covers descriptive statistics for all numeric variables, visual distributions, and correlation analysis.

A. Descriptive Statistics

The table below summarizes the central tendency and variation for all numeric variables in the integrated dataset.

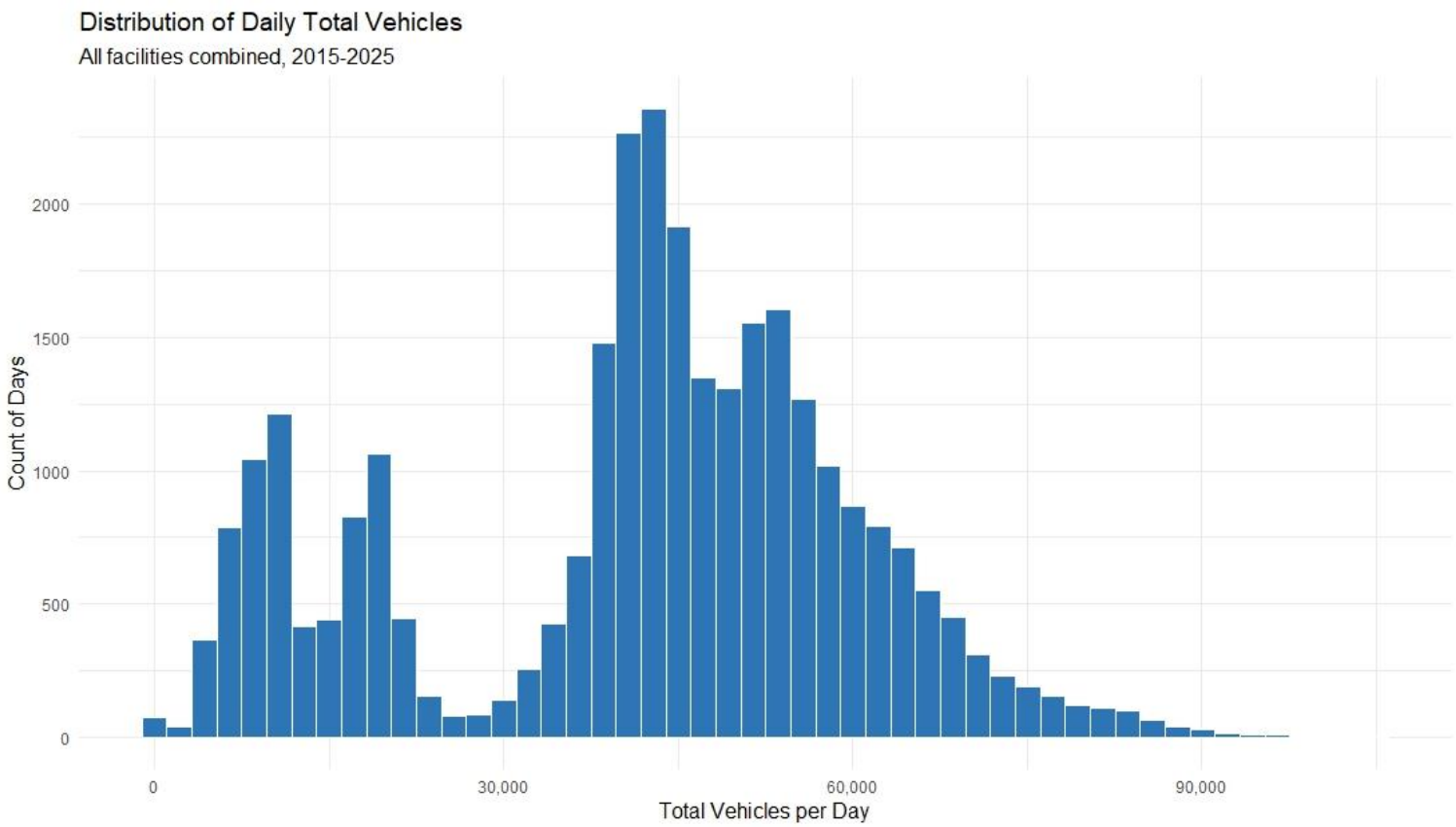
Variable	Mean	Median	Std Dev	Variance	Range	Mode
TOTAL VEHICLES	41,774	44,053	19,009	3.61e+08	105,092	40,999
CASH	5,356	5,058	4,024	1.62e+07	21,874	0
EZPASS	33,867	36,264	14,821	2.20e+08	80,904	33,591
VIOLATION	2,551	1,794	2,423	5.87e+06	20,759	0
Autos	38,168	40,752	16,972	2.88e+08	98,267	40,616
Small_T	1,411	966	1,555	2.42e+06	7,823	13
Large_T	1,326	305	2,274	5.17e+06	10,032	1
Buses	869	222	1,747	3.05e+06	7,802	43
ViolationRate	6.18%	4.19%	4.80%	0.00231	35.71%	0%
Avg_Speed (mph)	33.18	30.80	12.10	146.49	36.30	48.6
Delta (mph)	-9.40	-8.40	10.20	103.94	31.40	2.0
AWND (mph)	9.43	8.72	3.74	13.98	27.07	7.61
PRCP (in)	0.131	0.000	0.365	0.133	8.41	0
SNOW (in)	0.072	0.000	0.681	0.464	24.00	0
SNWD (in)	0.231	0.000	1.309	1.714	20.10	0
TMAX (°F)	65.17	66.00	18.54	343.59	89.00	87
TMIN (°F)	48.31	48.00	17.03	289.92	81.00	32
SevereEventCount	1.35	0.00	6.79	46.11	138.00	0
SevereFlag	0.129	0.000	0.109	0.0119	1.00	0
Gas_Price ($/gal)	2.888	2.696	0.696	0.485	2.753	2.341

Key Insights

- **Traffic volume is right-skewed:** Mean (41,774) is below median (44,053), pulled down by smaller facilities like Bayonne (~13,000/day). Most facility-days cluster between 35,000–65,000 vehicles.
- **Violations are heavily zero-inflated:** Both CASH and VIOLATION have a mode of 0, meaning many facility-days record no cash payments or violations at all. Yet VIOLATION has a max of 20,759 — outlier days deserve investigation.
- **ViolationRate average is 6.18% but can reach 35.7%:** Most days are within a normal 2–10% range, but the high maximum suggests specific facilities or events with abnormally high violation activity.
- **Weather variables are mostly zero:** PRCP, SNOW, and SNWD all have a median and mode of 0, confirming precipitation and snow are minority events — but when they occur, they can be extreme (PRCP max = 8.41 inches).
- **Gas prices span a wide range ($1.79–$4.55):** The SD of $0.70 and the range of $2.75 reflect three distinct price regimes over 2015–2025 — low prices (2015–2016, 2020), normal range ($2.50–$3.20), and the 2022 spike.
- **Delta is consistently negative (mean -9.40 mph):** Facilities operate below free-flow speed on average, confirming chronic congestion across the Port Authority network throughout the study period.

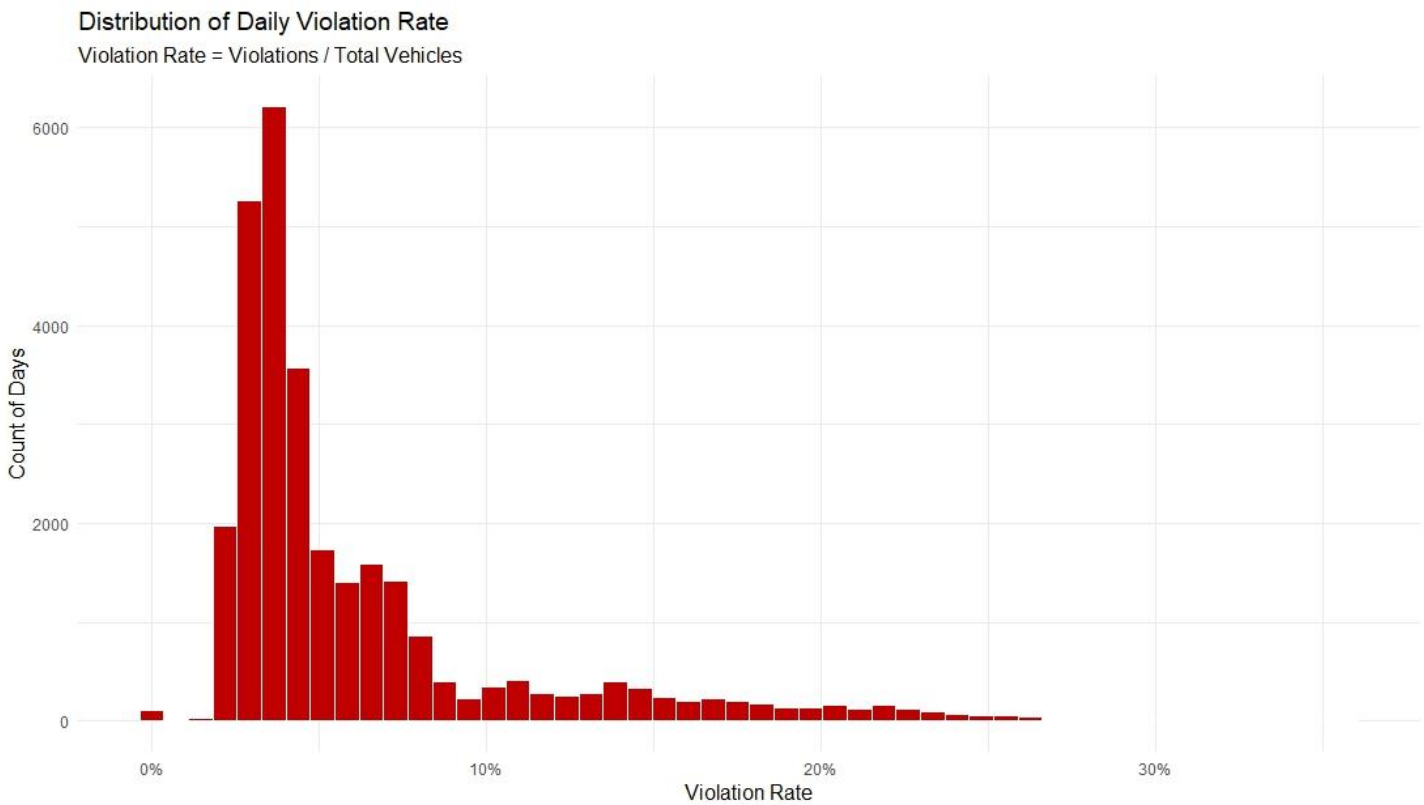
B. Visual Distributions

Figure 1 — Distribution of Daily Total Vehicles



- **Bimodal shape:** Two visible clusters — a lower group (~5,000–25,000) from smaller facilities like Bayonne and GWB PIP, and a dominant upper group (~35,000–65,000) from GWB Upper, GWB Lower, and Lincoln Tunnel.
- **Right tail extends to ~105,000:** Peak traffic days at major facilities push the upper bound, but these are rare events.
- **Takeaway:** Facility size drives the bimodal shape — any aggregate model should account for facility-level differences or be built per-facility.

Figure 2 — Distribution of Daily Violation Rate

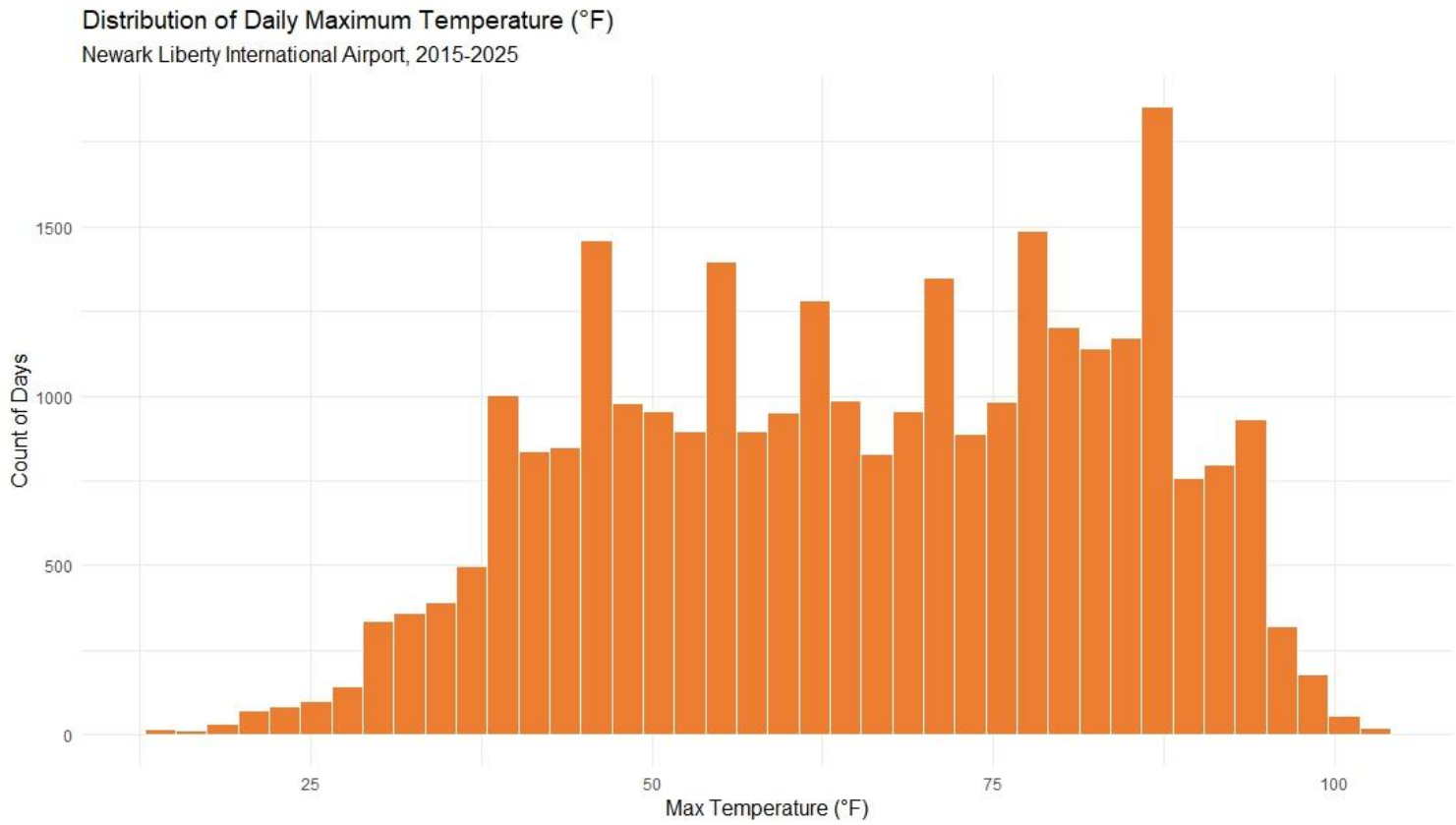


- **Strong right skew:** The bulk of facility-days fall between 2–8% violation rate. The long tail past 20% represents a small number of high-violation days worth flagging as anomalies.
- **Mode is 0%:** Reflects that many days — particularly at EZPass-dominant facilities — log very few or no violations.
- **Takeaway:** Violation rate behaves like an outlier-prone variable. Winsorizing or log-transforming it before modeling may improve model stability.

Figure 3 — Distribution of Daily Gas Prices

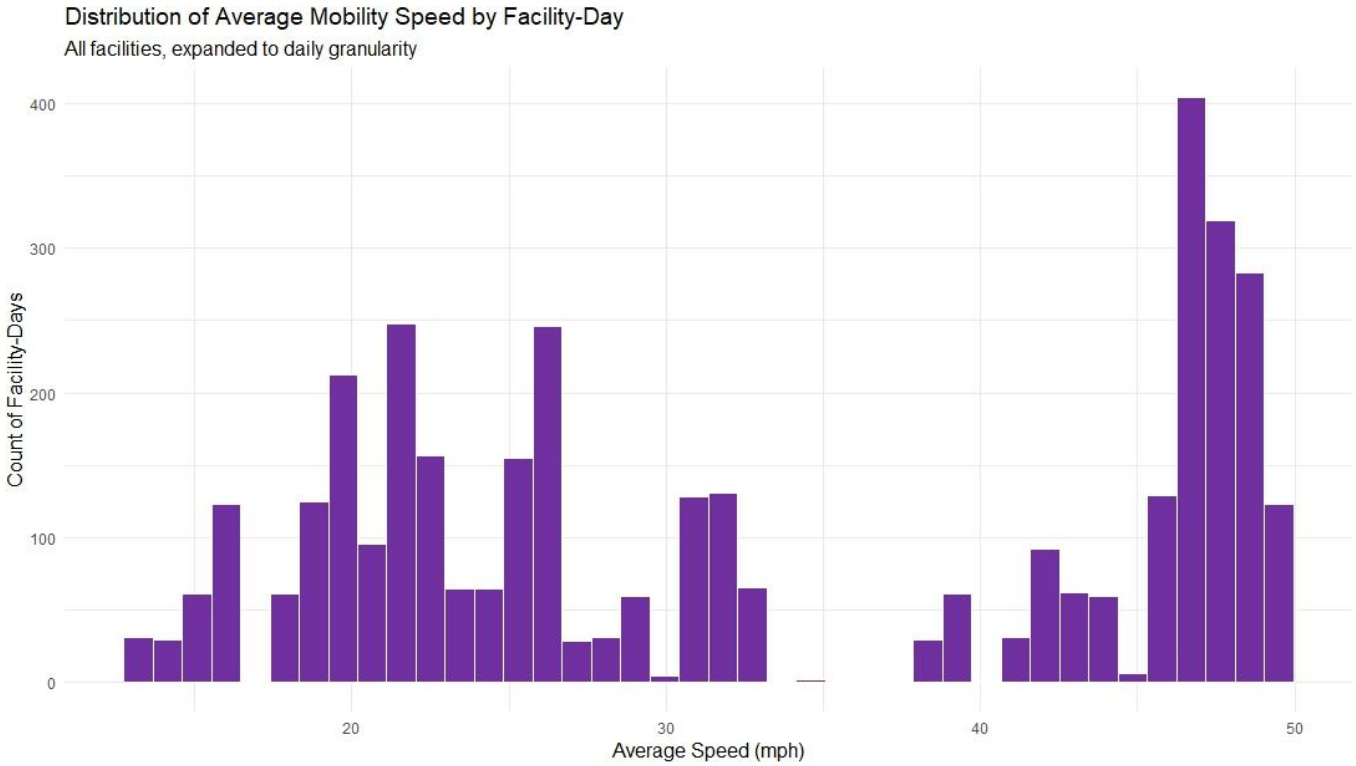
- **Multimodal with three distinct clusters:** Around $2.00–$2.30 (COVID-era 2020 and 2015–2016 lows), around $2.80–$3.00 (normal years), and $3.50–$4.50 (2022 spike).
- **High variance ($0.70 SD):** This wide spread gives the gas price variable strong potential as a predictor of traffic demand shifts over time.
- **Takeaway:** Gas price is not normally distributed — its multimodal nature reflects structural economic events, not random variation.

Figure 4 — Distribution of Daily Maximum Temperature



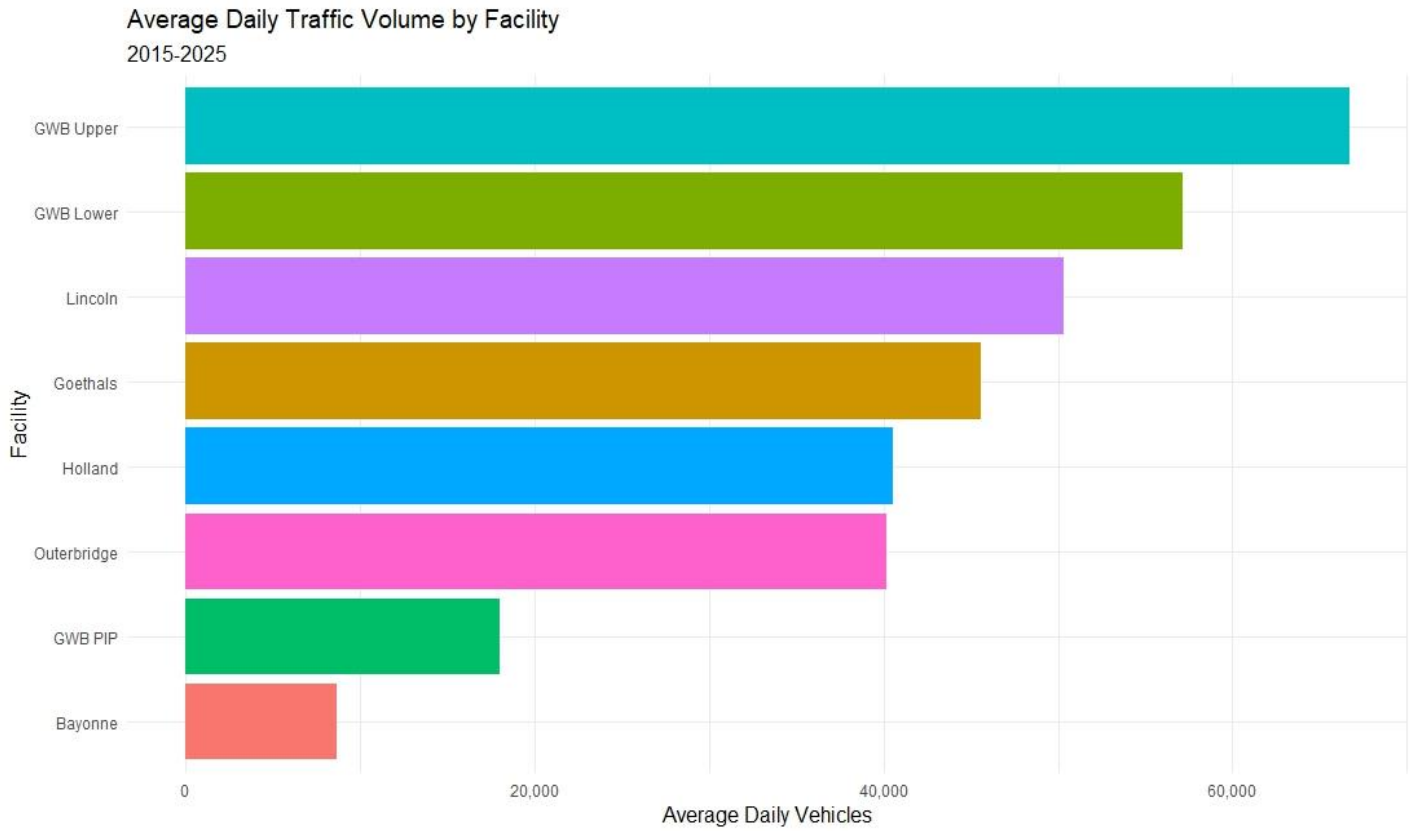
- **Bimodal seasonal pattern:** A winter cluster around 35–45°F and a summer cluster around 80–90°F reflect the four-season climate of NJ/NY.
- **Full range 14°F to 103°F:** Extreme cold and heat days are present and could suppress traffic volume at the tails.
- **Takeaway:** Temperature shows a non-linear relationship with traffic — moderate temperatures may correlate with higher traffic while extremes reduce it.

Figure 5 — Distribution of Average Mobility Speed



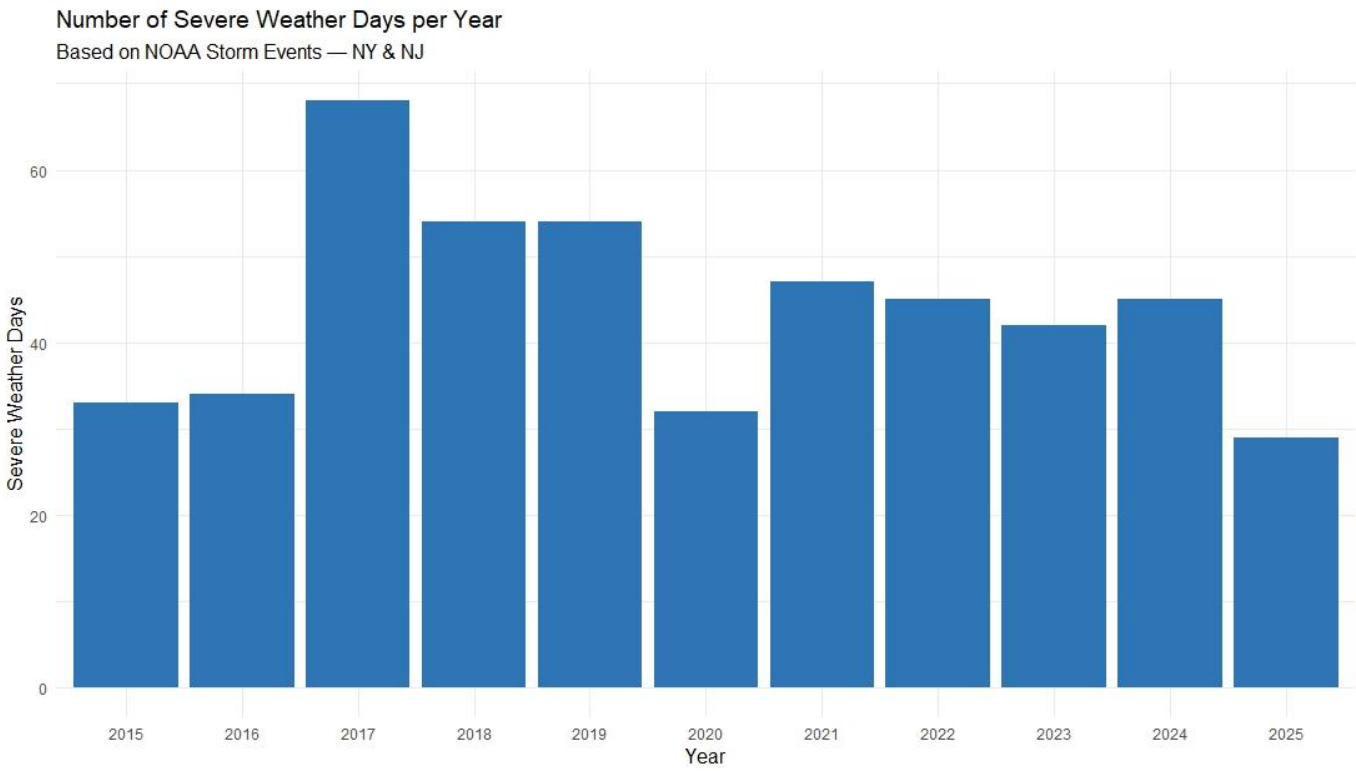
- **Two distinct speed regimes:** A congested cluster at 18–28 mph (tunnels: Holland, Lincoln) and a free-flowing cluster at 44–50 mph (bridges: Bayonne, Goethals, Outerbridge).
- **Gap between clusters:** Almost no facility-days fall between 32–40 mph, indicating facilities are either congested or free-flowing — rarely in between.
- **Takeaway:** Avg_Speed is a strong facility-type discriminator. It should be used as a feature in models but treated as facility-specific rather than network-wide.

Figure 6 — Average Daily Traffic Volume by Facility



- **GWB Upper is the busiest facility (~65,000/day):** Followed closely by GWB Lower (~57,000) and Lincoln Tunnel (~51,000). These three facilities account for the majority of total Port Authority traffic.
- **Bayonne is the least used (~13,000/day):** GWB PIP is the second least used at ~18,000. Both are candidates for targeted congestion pricing analysis.
- **Takeaway:** Any network-wide analysis must weight by facility — GWB alone represents nearly 3x the traffic of Bayonne.

Figure 7 — Severe Weather Days per Year

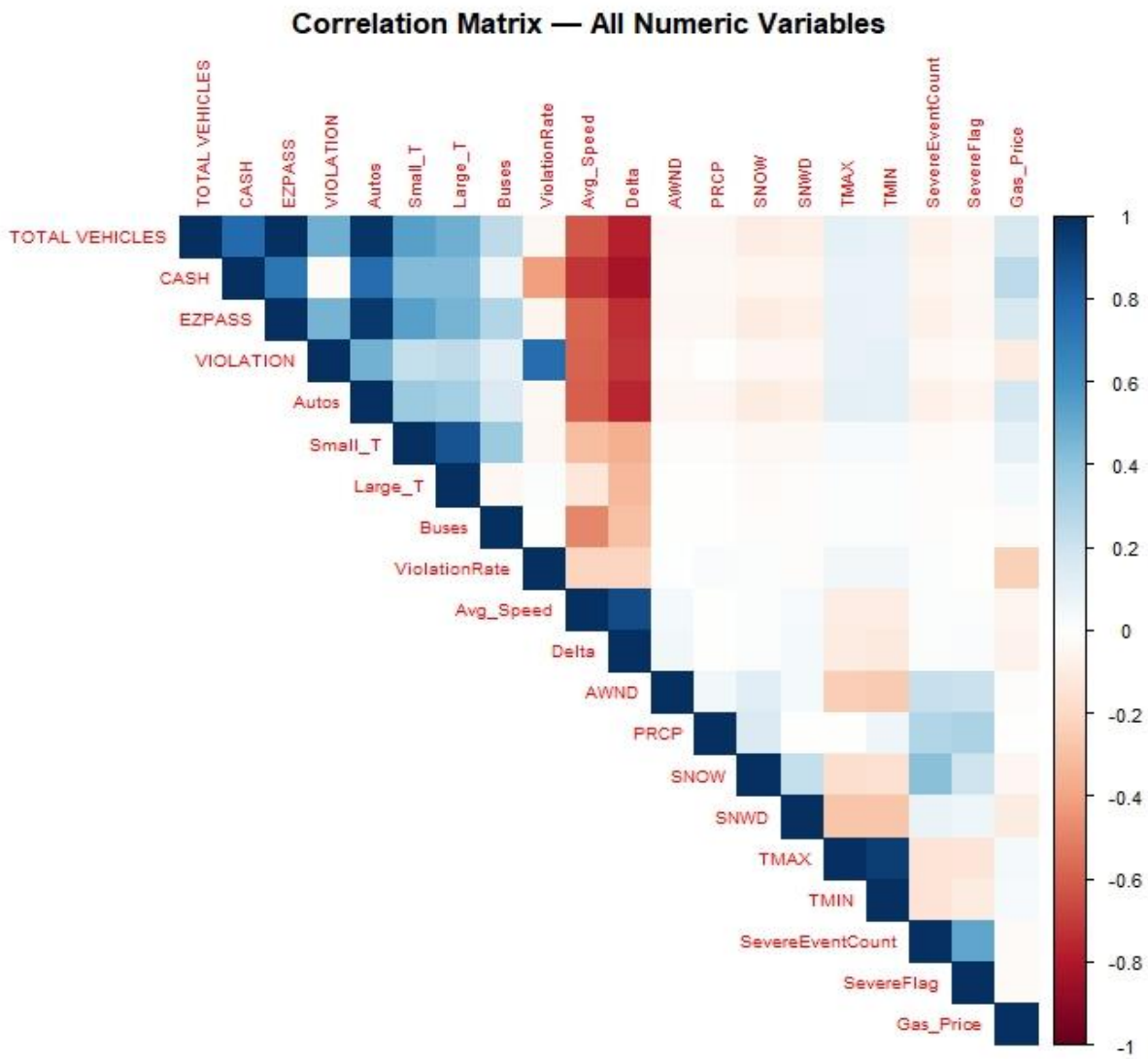


- **2017 was the worst year (69 severe days):** Followed by 2018 and 2019 (~54 each). This aligns with known active storm seasons in the NY/NJ region.
- **2020 had the fewest severe days (~32):** Partly reflects reduced human activity and potentially reduced storm reporting during COVID.
- **2025 appears low (~29):** Data only covers through September 2025 — not a full year.
- **Takeaway:** Severe weather days average ~44/year. Year-to-year variation is meaningful and should be a factor in annual traffic forecasting.

C. Correlation Analysis

Pearson correlation coefficients were computed across all numeric variables using pairwise complete observations. Results are visualized in a heatmap and ranked in the table below.

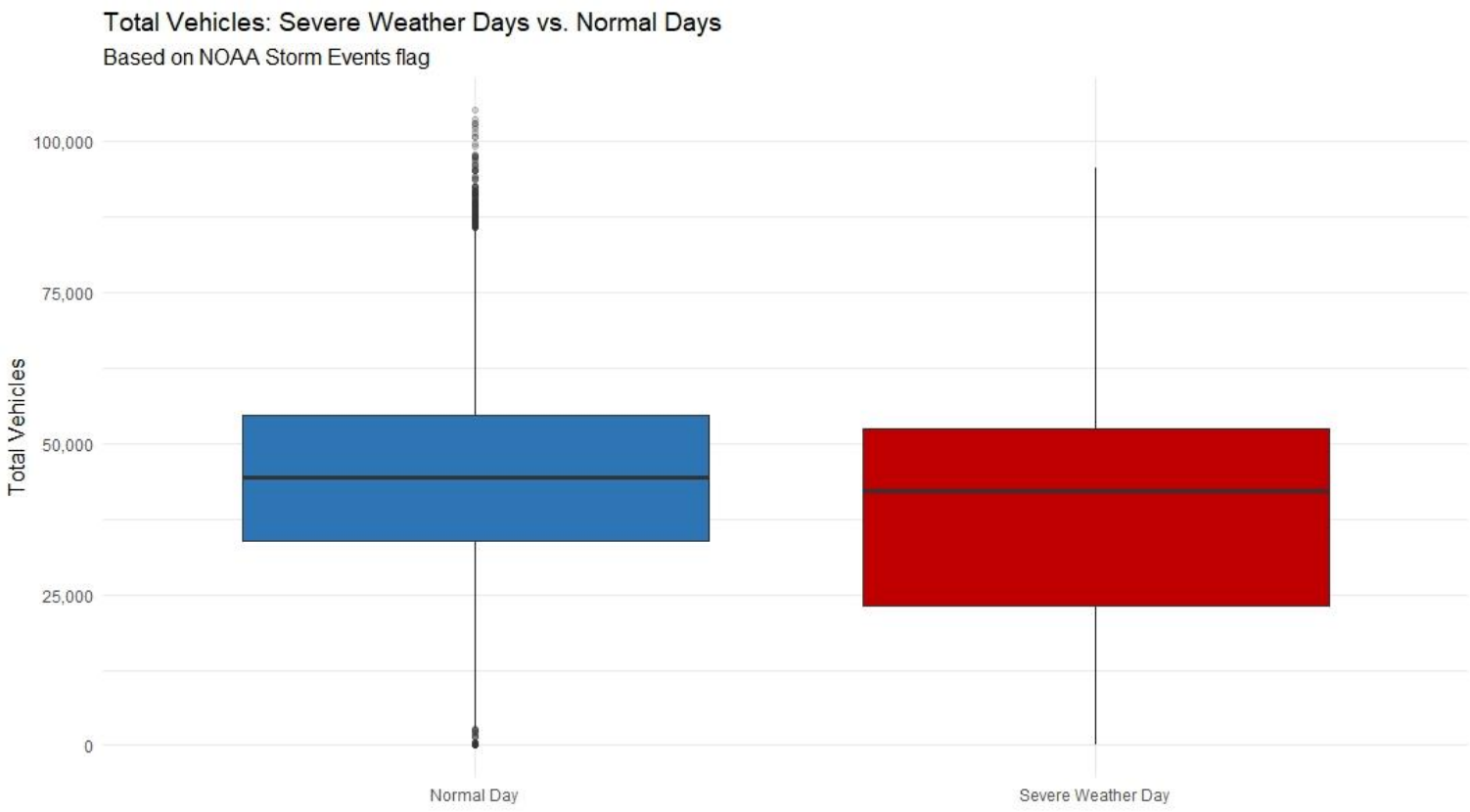
Figure 8 — Correlation Matrix Heatmap



Top 15 Strongest Correlations

Rank	Variable 1	Variable 2	Correlation (r)	Abs. Correlation
1	EZPASS	TOTAL VEHICLES	0.9928	0.9928
2	TOTAL VEHICLES	EZPASS	0.9928	0.9928
3	Autos	TOTAL VEHICLES	0.9778	0.9778
4	TOTAL VEHICLES	Autos	0.9778	0.9778
5	Autos	EZPASS	0.9689	0.9689
6	EZPASS	Autos	0.9689	0.9689
7	TMIN	TMAX	0.9416	0.9416
8	TMAX	TMIN	0.9416	0.9416
9	Delta	Avg_Speed	0.8990	0.8990
10	Avg_Speed	Delta	0.8990	0.8990
11	Large_T	Small_T	0.8688	0.8688
12	Small_T	Large_T	0.8688	0.8688
13	Delta	CASH	-0.8262	0.8262
14	CASH	Delta	-0.8262	0.8262
15	CASH	TOTAL VEHICLES	0.7765	0.7765

Figure 9 — Total Vehicles: Severe vs. Normal Days



On normal days, traffic volume is stable and follows a predictable routine, with most facility-days recording between 33,000 and 55,000 vehicles. On severe weather days, that consistency breaks down. While the median drops slightly to around 41,000 vehicles compared to 44,000 on normal days, the more important finding is the wider spread in the data. Some severe weather days record near-normal traffic because commuters have no choice but to travel, while others see volumes drop close to 25,000 as people choose to stay home. In short, severe weather does not always reduce traffic significantly, but it makes traffic volume far less predictable and harder to forecast.

Key Correlation Insights

- **EZPASS drives total traffic (r = 0.993):** EZPass transactions are nearly perfectly correlated with total vehicles — it is the dominant payment method across all facilities and effectively a proxy for total volume.
- **Automobiles dominate the vehicle mix (r = 0.978):** Autos correlate almost perfectly with TOTAL VEHICLES. Trucks, buses, and other classes contribute but are secondary drivers of total volume.
- **Temperature variables are highly collinear (r = 0.942):** TMAX and TMIN move together strongly. Only one should be included in regression models to avoid multicollinearity.
- **Speed and congestion are tightly linked (r = 0.899):** Avg_Speed and Delta are essentially measuring the same phenomenon. Delta adds no independent information beyond Avg_Speed and one should be dropped before modeling.
- **Congestion correlates negatively with cash payments (r = -0.826):** More congested facilities (lower Delta) tend to have higher cash usage. This reflects that tunnel facilities like Holland and Lincoln are both slower and more cash-reliant — a facility-type effect rather than a causal relationship.
- **Severe weather reduces traffic and increases variability:** The box plot shows that on severe weather days, the median traffic drops (~41,000 vs ~44,000 on normal days) and the IQR widens significantly. This means severe weather doesn't uniformly suppress traffic — its impact depends on storm timing, intensity, and facility location.

- **Weather and gas price show weak raw correlation with traffic:** PRCP, SNOW, TMAX, and Gas_Price show near-zero or weak correlation with TOTAL VEHICLES in the raw matrix. This does not mean they are irrelevant — their effects are likely non-linear and masked by facility-level variation. Controlled regression models are needed to isolate their true impact.

Section 3 — Data Cleaning

Data cleaning for this project was carried out in two phases. The first phase was performed during the initial dataset preparation stage, where Python was used to time-frame all datasets to the 2015–2025 study period and remove any rows that fell outside this range or contained empty values. This initial cleaning was essential to ensure that the Exploratory Data Analysis was based on consistent, complete, and relevant data — had it not been done at that stage, EDA insights such as trend analysis, seasonal patterns, and correlation results would have been distorted by irrelevant or incomplete records.

The second phase of cleaning, documented in this section, was performed using Python in Jupyter Notebook on the Anaconda platform. It addressed the remaining data quality issues identified during inspection of the five cleaned datasets.

A. Data Cleaning Summary

Dataset	Missing Values	Duplicates	Zero-Length Strings	Outliers (IQR)	Action Taken
traffic_daily_facility (29,334 rows)	None	None	None	Yes — VIOLATION (2,464), Buses (3,793), ViolationRate (3,343), Large_T (2,850), Small_T (2,147), CASH (407), Autos (603), TOTAL VEHICLES (197), EZPASS (27)	Outliers retained — legitimate traffic events. DATE converted to datetime.
mobility_daily_facility (5,975 rows)	None	None	None	None	DATE converted to datetime. No further action needed.
Weather_Daily (4,018 rows)	AWND: 9 rows SNWD: 3 rows	None	None	Yes — PRCP (784), SNWD (195), AWND (87), SNOW (145)	AWND and SNWD forward-filled. STATION and NAME dropped. Outliers retained — genuine weather events.
storm_events_daily (4,018 rows)	None	None	None	None	DATE converted to datetime. No further action needed.
gas_daily (3,895 rows)	None	None	None	None	DATE converted to datetime. No further action needed.

B. Key Cleaning Decisions

- **DATE conversion:** All five datasets stored DATE as a plain string. It was converted to Python datetime using pd.to_datetime() with the correct format specified per dataset. Without this, joins fail silently and produce empty results.

- **Forward-fill for weather missing values:** AWND had 9 missing rows and SNWD had 3 missing rows, all in 2025 due to a station reporting gap at Newark Airport. The dataset was sorted by date and ffill() applied so each missing day inherits the most recent valid reading. After this step both columns confirmed 0 missing values.

- **Dropped STATION and NAME from weather:** Both columns are constant across all 4,018 rows, always referring to Newark Liberty International Airport. They carry no analytical value and were dropped to keep the schema clean.

- **Outliers retained:** Outliers were identified using the IQR method across traffic and weather numeric columns. None were removed. Traffic outliers reflect genuine peak traffic events such as holidays and special events. Weather outliers represent real extreme precipitation and snowfall days. Moreover, these outliers will be addressed on a per-model basis during the modeling phase, as different models have different sensitivities to extreme values — for example, linear regression may require capping or transformation, while tree-based models such as Random Forest and XGBoost are robust to outliers by design..

- **No zero-length strings or duplicates:** All five datasets passed these checks cleanly, confirming the preparation done during initial dataset cleaning was effective.

C. Python Code – Tool used: Jupyter Notebook (Anaconda)

Step 1 — Load Libraries and Datasets

We start by importing pandas for data manipulation and numpy for numerical operations. All five cleaned CSV files are then loaded into separate dataframes.

```
import pandas as pd
import numpy as np

traffic  = pd.read_csv("traffic_daily_facility.csv")
mobility = pd.read_csv("mobility_daily_facility.csv")
weather  = pd.read_csv("Weather_Daily.csv")
storms   = pd.read_csv("storm_events_daily.csv")
gas      = pd.read_csv("gas_daily.csv")
```

Step 2 — Inspect Each Dataset Before Cleaning

Before making any changes, we print the shape, missing values, and duplicate row count for each dataset. This gives us a clear picture of what needs to be fixed.

```
for name, df in [("traffic", traffic), ("mobility", mobility),
                 ("weather", weather), ("storms", storms), ("gas", gas)]:
    print(f"Dataset: {name}")
    print(f"Shape: {df.shape}")
    print(f"Missing Values:\n{df.isnull().sum()}")
    print(f"Duplicate Rows: {df.duplicated().sum()}")
```

```
========================================
Dataset: traffic
Shape: (29334, 11)
Missing Values:
DATE              0
FAC_B             0
TOTAL VEHICLES    0
CASH              0
EZPASS            0
VIOLATION         0
Autos             0
Small_T           0
Large_T           0
Buses             0
ViolationRate     0
dtype: int64
Duplicate Rows: 0

========================================
Dataset: mobility
Shape: (5975, 4)
Missing Values:
DATE         0
FAC_B        0
Avg_Speed    0
Delta        0
dtype: int64
Duplicate Rows: 0

========================================
Dataset: weather
Shape: (4018, 9)
Missing Values:
STATION    0
NAME       0
DATE       0
AWND       9
PRCP       0
SNOW       0
SNWD       3
TMAX       0
TMIN       0
dtype: int64
Duplicate Rows: 0
```

```
========================================
Dataset: storms
Shape: (4018, 3)
Missing Values:
DATE                0
SevereEventCount    0
SevereFlag          0
dtype: int64
Duplicate Rows: 0

========================================
Dataset: gas
Shape: (3895, 2)
Missing Values:
DATE         0
Gas_Price    0
dtype: int64
Duplicate Rows: 0
```

Step 3 — Convert DATE to Datetime

All five datasets store DATE as a plain string, which means Python does not recognize it as a date. Without converting it to a proper datetime type, joins between datasets will fail silently and time-based operations will not work. Each dataset has a different raw date format so we specify the correct format string explicitly for each one.

```
traffic["DATE"]  = pd.to_datetime(traffic["DATE"],  format="%m/%d/%Y",
errors="coerce")
mobility["DATE"] = pd.to_datetime(mobility["DATE"], format="%Y-%m-%d",
errors="coerce")
weather["DATE"]  = pd.to_datetime(weather["DATE"],  format="%Y-%m-%d",
errors="coerce")
storms["DATE"]   = pd.to_datetime(storms["DATE"],   format="%m/%d/%Y",
errors="coerce")
gas["DATE"]      = pd.to_datetime(gas["DATE"],      format="%Y-%m-%d",
errors="coerce")
```

Step 4 — Drop Redundant Columns from Weather

The weather dataset contains two columns, STATION and NAME, that hold the same value in every single row — always the Newark Liberty International Airport station identifier. Since they never change, they carry no analytical value and would just add unnecessary columns to the final integrated dataset. We drop them here to keep the schema clean.

```
weather = weather.drop(columns=["STATION", "NAME"])
```

Step 5 — Forward-Fill Missing Values in Weather

Inspection revealed that AWND (average wind speed) had 9 missing rows and SNWD (snow depth) had 3 missing rows, all clustered in 2025. These gaps are caused by a reporting gap at the Newark Airport weather station rather than a structural data problem. We first sort the dataset by date to make sure forward-fill moves in the correct chronological direction, then apply ffill() which replaces each missing value with the most recent valid reading.

```
weather = weather.sort_values("DATE").reset_index(drop=True)
weather["AWND"] = weather["AWND"].ffill()
weather["SNWD"] = weather["SNWD"].ffill()

# Verify no missing values remain
print(weather[["AWND", "SNWD"]].isnull().sum())
```

Step 6 — Check for Zero-Length Strings

Zero-length strings are entries that appear blank but are stored as empty strings rather than NaN. Pandas does not recognize them as missing values, so they can cause silent failures during joins and group operations.

```
for name, df in [("traffic", traffic), ("mobility", mobility),
                 ("weather", weather), ("storms", storms), ("gas", gas)]:
    str_cols = df.select_dtypes(include=["object"]).columns
    for col in str_cols:
        zls = (df[col].astype(str).str.strip() == "").sum()
        print(f"{name}.{col}: {zls} zero-length strings")
```

```
traffic.FAC_B: 0 zero-length strings
mobility.FAC_B: 0 zero-length strings
```

Step 7 — Identify Outliers Using the IQR Method

We use the Interquartile Range method to flag statistical outliers — any value below Q1 minus 1.5 times the IQR or above Q3 plus 1.5 times the IQR is considered an outlier. This step only identifies and documents the outliers. No values are removed. The outliers found in traffic columns such as VIOLATION (2,464) and Buses (3,793) represent genuine peak traffic events like holidays and major incidents. The outliers in weather columns such as PRCP (784) and SNOW (145) represent real extreme weather days. Removing them would make the model blind to the exact conditions it is supposed to predict. Outlier treatment will be applied on a per-model basis during the modeling phase, since different models handle extreme values differently.

```
for name, df in [("traffic", traffic), ("weather", weather)]:
    num_cols = df.select_dtypes(include=[np.number]).columns
    for col in num_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        n = ((df[col] < Q1 - 1.5 * IQR) | (df[col] > Q3 + 1.5 * IQR)).sum()
        if n > 0:
            print(f"{name}.{col}: {n} outliers identified — retained")
```

```
traffic.TOTAL VEHICLES: 197 outliers identified — retained
traffic.CASH: 407 outliers identified — retained
traffic.EZPASS: 27 outliers identified — retained
traffic.VIOLATION: 2464 outliers identified — retained
traffic.Autos: 603 outliers identified — retained
traffic.Small_T: 2147 outliers identified — retained
traffic.Large_T: 2850 outliers identified — retained
traffic.Buses: 3793 outliers identified — retained
traffic.ViolationRate: 3343 outliers identified — retained
weather.AWND: 87 outliers identified — retained
weather.PRCP: 784 outliers identified — retained
weather.SNOW: 145 outliers identified — retained
weather.SNWD: 195 outliers identified — retained
```

Step 8 — Final Verification

After all cleaning steps are complete, we run a final check to confirm that zero missing values remain across all five datasets. This validates that the cleaning process is fully complete and that all datasets are in a consistent state ready for integration.

```
for name, df in [("traffic", traffic), ("mobility", mobility),
                 ("weather", weather), ("storms", storms), ("gas", gas)]:
    print(f"{name}: {df.isnull().sum().sum()} missing values remaining")
```

```
Final Missing Value Check:
  traffic: 0 missing values remaining
  mobility: 0 missing values remaining
  weather: 0 missing values remaining
  storms: 0 missing values remaining
  gas: 0 missing values remaining
```

Step 9 — Save Cleaned Datasets

Once all cleaning steps are verified, the five cleaned dataframes are saved as new CSV files. These are the files that will be used in the SQL data integration step.

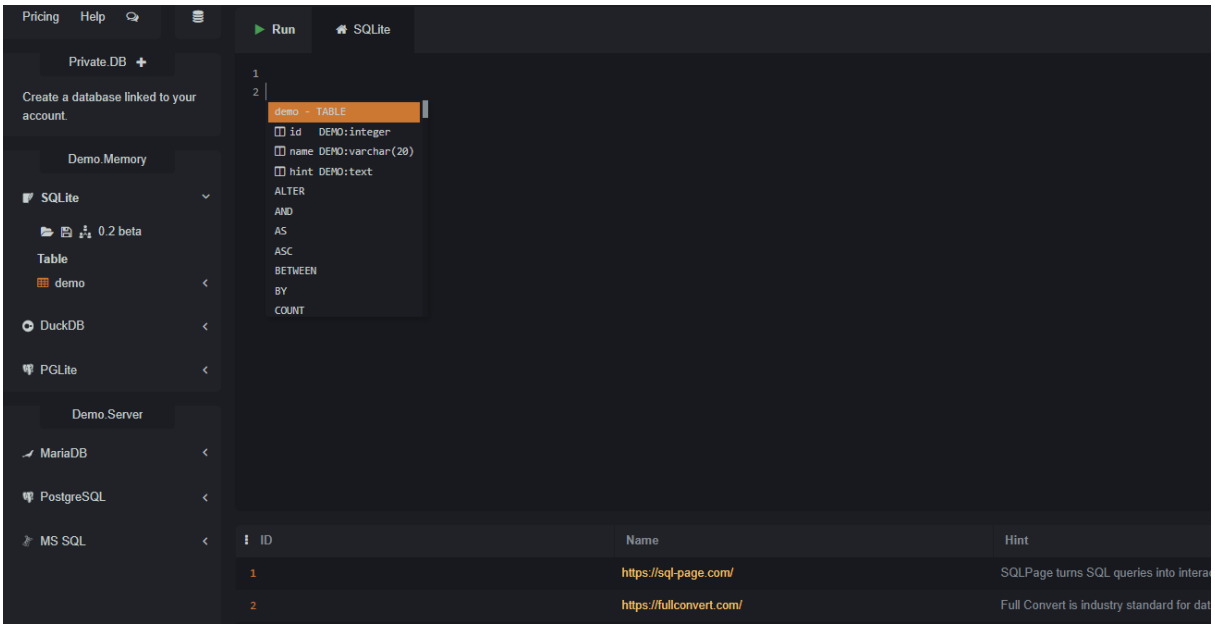
```
traffic.to_csv("traffic_daily_facility_clean.csv",   index=False)
mobility.to_csv("mobility_daily_facility_clean.csv", index=False)
weather.to_csv("Weather_Daily_clean.csv",            index=False)
storms.to_csv("storm_events_daily_clean.csv",        index=False)
gas.to_csv("gas_daily_clean.csv",                    index=False)

print("All cleaned datasets saved successfully.")
```

Section 4 — Data Integration Techniques

1. Tool Used

Data integration was performed using SQL on SQLiteOnline (sqliteonline.com), a free browser-based online SQL IDE requiring no installation. The five cleaned CSV files from the Python cleaning

phase were uploaded as individual tables and a single SQL query was executed to join all five into one master dataset, which was then exported as master_dataset.csv.



2. Integration Technique and Automation

The integration was performed using a series of four SQL LEFT JOINs, all chained into a single query. The traffic table served as the base since it defines the grain of the master dataset — one row per facility per day. All other tables were joined onto it. Using a single query automates the entire process — if any source dataset is updated, the same query can be re-run on the new files to regenerate the master dataset immediately with no manual steps.

Left Table	Right Table	Join Key(s)	Join Type
traffic	mobility	DATE + FAC_B (composite)	LEFT JOIN
traffic	weather	DATE	LEFT JOIN
traffic	storms	DATE	LEFT JOIN
traffic	gas	DATE	LEFT JOIN

```
SELECT
    t.date, t.fac_b, t.total_vehicles, t.cash, t.ezpass,
    t.violation, t.autos, t.small_t, t.large_t, t.buses,
    t.violationrate, m.avg_speed, m.delta,
    w.awnd, w.prcp, w.snow, w.snwd, w.tmax, w.tmin,
    s.severeeventcount, s.severeflag,
    g.gas_price
FROM traffic t
LEFT JOIN mobility m ON t.date = m.date AND t.fac_b = m.fac_b
LEFT JOIN weather w  ON t.date = w.date
LEFT JOIN storms s   ON t.date = s.date
LEFT JOIN gas g      ON t.date = g.date;
```

```sql
SELECT
    t.DATE,
    t.FAC_B,
    t."TOTAL_VEHICLES",
    t.CASH,
    t.EZPASS,
    t.VIOLATION,
    t.Autos,
    t.Small_T,
    t.Large_T,
    t.Buses,
    t.ViolationRate,
    m.Avg_Speed,
    m.Delta,
    w.AWND,
    w.PRCP,
    w.SNOW,
    w.SNWD,
    w.TMAX,
    w.TMIN,
    s.SevereEventCount,
    s.SevereFlag,
    g.Gas_Price
FROM traffic t
LEFT JOIN mobility m ON t.DATE = m.DATE AND t.FAC_B = m.FAC_B
LEFT JOIN weather w ON t.DATE = w.DATE
LEFT JOIN storms s ON t.DATE = s.DATE
LEFT JOIN gas g ON t.DATE = g.DATE;
```

date	fac_b	total_vehi…	cash	ezp…	vio…	autos	sm…	lar…	bu…	vio…	av…	delta	awnd	prcp	snow	snwd	tmax	tmin	sev…
2015-01-01	Bay…	3991	673	32…	96	39…	20	16	7	0.…	NULL	NULL	12…	0	0	0	39	20	0
2015-01-01	GW…	38009	68…	29…	21…	37…	13	4	148	0.…	NULL	NULL	12…	0	0	0	39	20	0
2015-01-01	GW…	13351	16…	11…	410	13…	7	0	16	0.…	NULL	NULL	12…	0	0	0	39	20	0
2015-01-01	GW…	50512	15…	34…	10…	47…	488	16…	402	0.…	NULL	NULL	12…	0	0	0	39	20	0
2015-01-01	Goet…	28277	59…	21…	602	27…	198	209	52	0.…	NULL	NULL	12…	0	0	0	39	20	0
2015-01-01	Holl…	31786	81…	22…	682	31…	231	1	245	0.…	NULL	NULL	12…	0	0	0	39	20	0

3. Data Relationships and Referential Integrity

- **DATE as universal key:** All five datasets share DATE as a common field. Consistent formatting was enforced during Python cleaning so no join failures occur due to format mismatches.
- **Composite key for mobility:** Mobility is the only facility-level dataset. Joining on DATE + FAC_B enforces a strict one-to-one match and prevents speed data from one facility being incorrectly assigned to another.
- **LEFT JOINs preserve base table:** Every traffic row is retained in the final output regardless of whether external data exists for that date. No traffic records are lost due to incomplete coverage in external datasets.
- **Known NULLs documented:** Avg_Speed and Delta are NULL before 2024 (mobility only covers 2024 onwards). Gas_Price is NULL for Jan–Apr 2015 (EIA data gap). These are known coverage limitations, not join errors.
- **No row duplication:** Weather, storms, and gas each have one row per date. Joining them onto traffic (multiple rows per date) produces exactly one match per traffic row, preserving the one-row-per-facility-per-day grain throughout.

3. Glimpse at the Final Dataset:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
1	date	fac_b	total_vehi	cash	ezpass	violation	autos	small_t	large_t	buses	violationr	avg_speec	delta	awnd	prcp	snow	snwd	tmax	tmin	severe_ev	severe_fla	gas_price
2	1/1/2015	Bayonne	3991	673	3222	96	3948	20	16	7	0.024054	null	null	12.53	0	0	0	39	20	0	0	null
3	1/1/2015	GWB Lowe	38009	6818	29071	2120	37844	13	4	148	0.055776	null	null	12.53	0	0	0	39	20	0	0	null
4	1/1/2015	GWB PIP	13351	1618	11323	410	13328	7	0	16	0.030709	null	null	12.53	0	0	0	39	20	0	0	null
5	1/1/2015	GWB Uppe	50512	15238	34261	1013	47930	488	1692	402	0.020055	null	null	12.53	0	0	0	39	20	0	0	null
6	1/1/2015	Goethals	28277	5934	21741	602	27818	198	209	52	0.021289	null	null	12.53	0	0	0	39	20	0	0	null
7	1/1/2015	Holland	31786	8111	22993	682	31309	231	1	245	0.021456	null	null	12.53	0	0	0	39	20	0	0	null
8	1/1/2015	Lincoln	36025	8098	27102	825	32538	340	97	3050	0.022901	null	null	12.53	0	0	0	39	20	0	0	null
9	1/1/2015	Outerbrid	29349	3881	24574	894	28886	163	228	72	0.030461	null	null	12.53	0	0	0	39	20	0	0	null
10	1/2/2015	Bayonne	7506	846	6468	192	6888	284	286	48	0.02558	null	null	10.51	0	0	0	44	31	0	0	null
11	1/2/2015	GWB Lowe	51005	9196	39794	2015	50673	84	13	235	0.039506	null	null	10.51	0	0	0	44	31	0	0	null
12	1/2/2015	GWB PIP	18284	2102	15674	508	18249	8	0	27	0.027784	null	null	10.51	0	0	0	44	31	0	0	null
13	1/2/2015	GWB Uppe	59426	14298	43739	1390	50360	3147	5215	704	0.02339	null	null	10.51	0	0	0	44	31	0	0	null
14	1/2/2015	Goethals	37857	6556	30255	1046	34461	1227	1856	313	0.02763	null	null	10.51	0	0	0	44	31	0	0	null
15	1/2/2015	Holland	40554	9160	30389	1005	38893	1139	21	501	0.024782	null	null	10.51	0	0	0	44	31	0	0	null
16	1/2/2015	Lincoln	48072	8487	38430	1155	39215	2147	425	6285	0.024026	null	null	10.51	0	0	0	44	31	0	0	null
17	1/2/2015	Outerbrid	37478	4383	31755	1340	35308	868	1199	103	0.035754	null	null	10.51	0	0	0	44	31	0	0	null
18	1/3/2015	Bayonne	5265	816	4340	109	5057	136	43	29	0.020703	null	null	6.93	0.79	0.6	0	39	26	3	1	null
19	1/3/2015	GWB Lowe	45578	8406	35152	2020	45316	60	17	185	0.04432	null	null	6.93	0.79	0.6	0	39	26	3	1	null
20	1/3/2015	GWB PIP	14952	1680	12845	427	14925	11	0	16	0.028558	null	null	6.93	0.79	0.6	0	39	26	3	1	null
21	1/3/2015	GWB Uppe	57462	15573	40715	1174	52537	1527	2809	589	0.020431	null	null	6.93	0.79	0.6	0	39	26	3	1	null
22	1/3/2015	Goethals	34662	6786	27033	843	33448	562	556	96	0.024321	null	null	6.93	0.79	0.6	0	39	26	3	1	null
23	1/3/2015	Holland	38062	9040	28272	750	37197	554	7	304	0.019705	null	null	6.93	0.79	0.6	0	39	26	3	1	null
24	1/3/2015	Lincoln	44475	8428	34968	1079	39911	1001	187	3376	0.024261	null	null	6.93	0.79	0.6	0	39	26	3	1	null
25	1/3/2015	Outerbrid	34467	4581	28766	1120	33291	478	610	88	0.032495	null	null	6.93	0.79	0.6	0	39	26	3	1	null
26	1/4/2015	Bayonne	4548	660	3777	111	4463	43	29	13	0.024406	null	null	10.29	0.27	0	0	56	39	7	1	null
27	1/4/2015	GWB Lowe	45141	8393	34980	1768	44845	38	8	250	0.039166	null	null	10.29	0.27	0	0	56	39	7	1	null
28	1/4/2015	GWB PIP	17101	1839	14778	484	17076	5	1	19	0.028302	null	null	10.29	0.27	0	0	56	39	7	1	null
29	1/4/2015	GWB Uppe	56137	15798	39200	1141	52502	658	2517	460	0.020325	null	null	10.29	0.27	0	0	56	39	7	1	null
30	1/4/2015	Goethals	34038	6820	26407	811	33245	300	385	108	0.023826	null	null	10.29	0.27	0	0	56	39	7	1	null
31	1/4/2015	Holland	36345	8797	26829	719	35865	223	4	253	0.019783	null	null	10.29	0.27	0	0	56	39	7	1	null
32	1/4/2015	Lincoln	41784	7657	33364	763	38289	434	114	2947	0.018261	null	null	10.29	0.27	0	0	56	39	7	1	null

Section 5 — Data Sampling

Data sampling was performed using Python (pandas) in Jupyter Notebook on the Anaconda platform. The master dataset (Traffic_by_Facility_FINAL.csv) containing 29,445 rows and 22 columns was used as the source. Four sampling techniques were applied to demonstrate different approaches suitable for this time series, multi-facility dataset.

```
import pandas as pd
import numpy as np

# — Load Master Dataset —
df = pd.read_csv("Traffic by Facility_FINAL.csv")
df['date'] = pd.to_datetime(df['date'], format='%m/%d/%Y', errors='coerce')

print(f"Full dataset shape: {df.shape}")
```

```
Full dataset shape: (29445, 22)
```

Technique 1 — Random Sampling (10%)

A random 10% sample is drawn from the full dataset using a fixed random seed (random_state=42) to ensure the results are reproducible. This technique is useful for quick model prototyping and initial exploratory analysis where a smaller representative subset is needed without any specific structure.

```
random_sample = df.sample(frac=0.10, random_state=42)
print(f"Random Sample (10%): {random_sample.shape}")
print(random_sample[['date', 'fac_b', 'total_vehicles']].head())
```

```
Random Sample (10%): (2944, 22)
            date        fac_b  total_vehicles
10930 2018-10-02  Outerbridge           38431
18955 2021-07-02     Goethals           53300
29383 2025-05-23    GWB Lower           66040
24845 2023-08-29      Lincoln           50420
29339 2025-05-16  Outerbridge           43611
```

Output: 2,944 rows drawn randomly across all facilities and years. The sample spans multiple facilities and dates confirming it is well distributed across the full dataset.

Technique 2 — Stratified Sampling (10% per Facility)

Stratified sampling draws 10% from each facility group independently. This is the most appropriate technique for this dataset because facilities have different total row counts — GWB Upper and Lower each have 3,820 rows while GWB PIP only has 2,747. A pure random sample would underrepresent smaller facilities. Stratified sampling guarantees every facility is proportionally present in the sample, which is critical for building fair facility-level models.

```
stratified_sample = df.groupby('fac_b', group_keys=False).apply(
    lambda x: x.sample(frac=0.10, random_state=42)
)
print(f"Stratified Sample (10% per facility): {stratified_sample.shape}")
print(stratified_sample['fac_b'].value_counts())
```

```
Stratified Sample (10% per facility): (2945, 22)
fac_b
GWB Lower      382
GWB Upper      382
Goethals       382
Holland        382
Lincoln        382
Outerbridge    382
Bayonne        378
GWB PIP        275
Name: count, dtype: int64
```

Output: 2,945 rows with all 8 facilities represented proportionally. Larger facilities contribute 382 rows each while GWB PIP contributes 275, exactly reflecting the 10% split from each group.

Technique 3 — Systematic Sampling (Every 10th Row)

Systematic sampling selects every 10th row from the dataset after it is sorted by date. This technique is particularly well suited for time series data because it preserves the temporal spacing of the original dataset — the sample is spread evenly across the full 2015–2024 date range rather than clustering in any particular period. This makes it useful for training models that need to learn from patterns distributed across the full study period.

```
systematic_sample = df.iloc[::10].reset_index(drop=True)
print(f"Systematic Sample (every 10th row): {systematic_sample.shape}")
print(systematic_sample[['date', 'fac_b', 'total_vehicles']].head())
```

```
Systematic Sample (every 10th row): (2945, 22)
        date     fac_b  total_vehicles
0 2015-01-01   Bayonne            3991
1 2015-01-02   GWB PIP           18284
2 2015-01-03  Goethals           34662
3 2015-01-04   Lincoln           41784
4 2015-01-06   Bayonne            7561
```

Output: 2,945 rows evenly spaced across the full dataset. The output confirms rows from 2015 through to recent dates, showing the sample covers the entire study period without temporal clustering.

Technique 4 — Time-Based Sampling (Year 2019)

Time-based sampling extracts all records from a specific year — in this case 2019. The year 2019 was chosen as it represents a normal, stable operating year: it is post the major infrastructure changes of earlier years, pre-COVID disruptions of 2020, and pre the 2022 fuel price spike. This makes it a clean baseline year that is useful for training and evaluating models where the goal is to understand typical traffic behaviour without the distortion of extreme external events.

```
time_sample = df[df['date'].dt.year == 2019].reset_index(drop=True)
print(f"Time-Based Sample (Year 2019): {time_sample.shape}")
print(time_sample[['date', 'fac_b', 'total_vehicles']].head())
```

```
Time-Based Sample (Year 2019): (2916, 22)
        date      fac_b  total_vehicles
0 2019-01-01    Bayonne            4659
1 2019-01-01  GWB Lower           50459
2 2019-01-01    GWB PIP           15716
3 2019-01-01  GWB Upper           57014
4 2019-01-01   Goethals           41144
```

Output: 2,916 rows covering all 8 facilities across the full calendar year 2019. All facilities are present from January 1st confirming complete coverage for the selected year.

Section 7 — Final Dataset

The final integrated dataset – ***Traffic by Facility_FINAL.csv***, is the product of merging two internal and three external datasets using SQL LEFT JOINs. It is ready for analysis, modeling, and forecasting.

A. Dataset Overview

Characteristic	Value
File Name	Traffic_by_Facility_FINAL.csv
Total Rows	29,445
Total Columns	22
Date Range	January 1, 2015 — May 31, 2025
Granularity	One row per facility per day
Number of Facilities	8
Facilities Covered	Bayonne, GWB Lower, GWB PIP, GWB Upper, Goethals, Holland, Lincoln, Outerbridge
Missing Values	avg_speed and delta: 25,715 NULLs (mobility covers 2024 onwards only) gas_price: 982 NULLs (EIA data gap Jan–Apr 2015)
Internal Datasets	Traffic Daily Facility, Mobility Daily Facility
External Datasets	NOAA Weather, NOAA Storm Events, EIA Gas Prices

B. Column Descriptions

Column	Type	Description
date	Date	Date of observation (daily)
fac_b	Text	Facility name (8 Port Authority facilities)
total_vehicles	Integer	Total vehicles crossing the facility that day
cash	Integer	Vehicles that paid by cash
ezpass	Integer	Vehicles that paid via EZPass
violation	Integer	Vehicles recorded as toll violations
autos	Integer	Number of automobiles
small_t	Integer	Small trucks
large_t	Integer	Large trucks
buses	Integer	Buses
violationrate	Float	Derived: violation / total_vehicles
avg_speed	Float	Average mobility speed at facility (mph) — from 2024 onwards
delta	Float	Deviation from free-flow speed (mph) — from 2024 onwards
awnd	Float	Average daily wind speed (mph) — Newark Airport
prcp	Float	Daily precipitation (inches)
snow	Float	Daily snowfall (inches)
snwd	Float	Snow depth on ground (inches)
tmax	Integer	Maximum daily temperature (°F)
tmin	Integer	Minimum daily temperature (°F)
severe_event_count	Integer	Number of severe weather events in NY/NJ that day
severe_flag	Binary	1 = severe weather day, 0 = normal day
gas_price	Float	East Coast PADD 1 retail gas price ($/gallon)

C. Key Characteristics

- **Scope:** Covers 10+ years of daily traffic data (January 2015 to May 2025) across all 8 Port Authority crossings — bridges and tunnels connecting New York and New Jersey.
- **Granularity:** One row per facility per day. With 8 facilities across ~3,680 days, the dataset contains 29,445 rows — complete and uninterrupted except for known coverage gaps.
- **Target variable:** total_vehicles is the primary variable of interest for forecasting and regression models. It ranges from 1 to 105,093 with a mean of 41,774 vehicles per facility per day.
- **Payment split:** EZPass dominates across all facilities, accounting for the vast majority of transactions. Cash payments are declining over the study period, reflecting the shift toward electronic tolling.
- **Violation rate:** Averages 6% across the dataset with a maximum of 36%, indicating most days are within a normal range but certain facility-days have abnormally high violation activity worth investigating.
- **Weather integration:** Daily weather from Newark Airport (temperature, precipitation, wind, snow) and NOAA storm event flags are included, covering the full 2015–2025 period with no gaps after forward-filling.
- **Severe weather:** 3,676 out of 29,445 facility-days are flagged as severe weather days (approximately 12.5%), providing enough severe-day observations for meaningful weather impact modeling.
- **Gas price range:** Spans $1.79 to $4.55 per gallon, capturing low-price periods (2015–2016, COVID 2020), normal years, and the 2022 fuel price spike — giving the model strong economic variation to learn from.
- **Known NULLs:** avg_speed and delta are NULL for dates before 2024 (mobility data coverage limitation). gas_price is NULL for January to April 2015 (EIA data gap). These are documented limitations, not data errors.